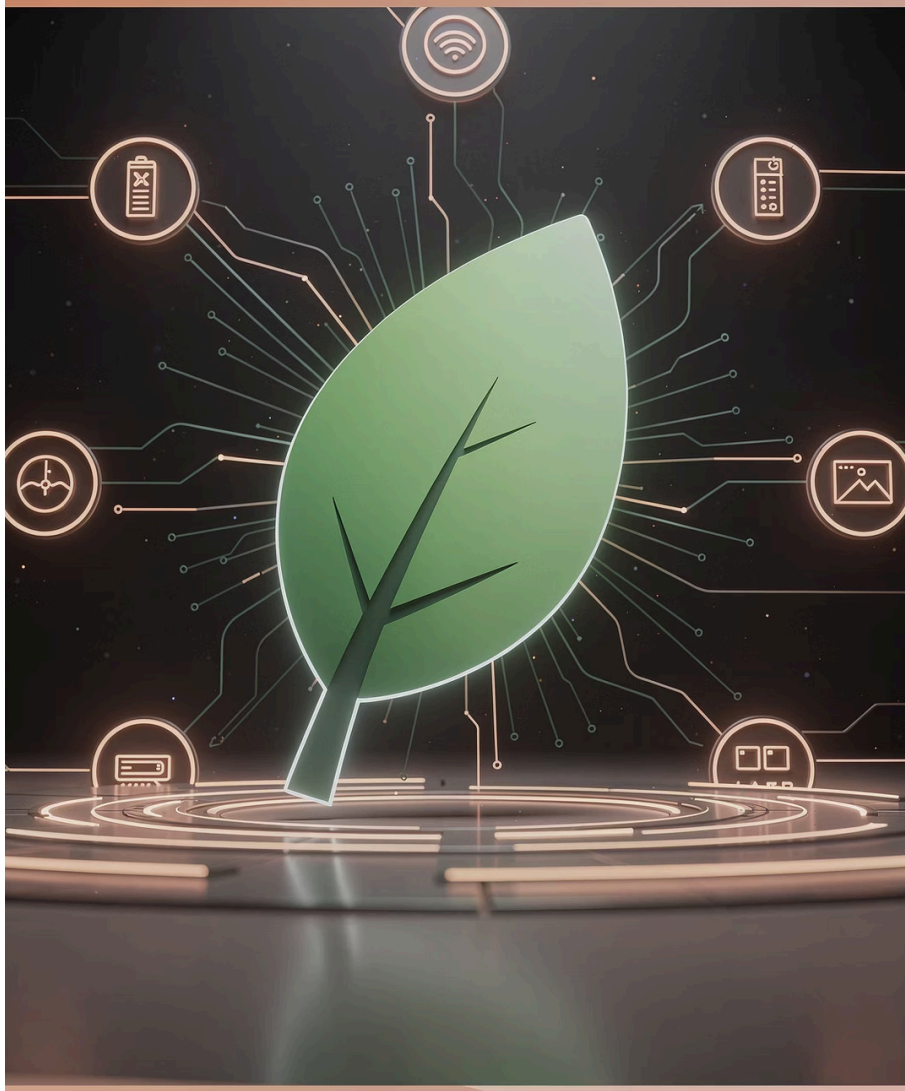
MongoDB

Getting started with the world's leading NoSQL database



Installation Setup

Install **2 things** to get started:

MongoDB Community Server

Download here

MongoDB Shell

Download here

🪟 Windows

Full guide: YouTube Tutorial

🍎 Mac

Full guide: YouTube Tutorial

Video shows v6.0 — install **v7.0** (latest).
Commands remain the same.

mongosh

The modern MongoDB Shell for interacting with your database

Start mongosh

Just type `mongosh` in your terminal.

`show dbs`
List all databases

`use <db>`
Switch to or create a database

`show collections`
List collections in the current database

`db.createCollection("name")`
Create a collection

`db.<collection>.insertOne({})`
Insert a document

`db.<collection>.find()`
Query all documents

`db.<collection>.updateOne()`
Update a document

`db.<collection>.deleteOne()`
Delete a document

`exit`
Exit the shell

mongosh & JavaScript

mongosh is a full JavaScript interpreter — you can run JS directly in the shell.

`Math.round(2.6)`

Use built-in JS math functions

`let x = 10; x * 2`

Declare variables and evaluate expressions

`new Date()`

Use JS Date objects

`[1,2,3].forEach(n => print(n))`

Use array methods and arrow functions

`function greet(name) { return "Hello, " + name }`

Define and call functions

`db.users.find().forEach(doc => print(doc.name))`

Combine JS with MongoDB queries

Any valid JavaScript expression can be typed directly into the mongosh prompt.

JSON vs JavaScript Object

They look almost identical — but have key differences.

JavaScript Object

```
{
  name: "Alice",
  age: 30,
  active: true
}
```

- Keys don't need quotes
- Values can be functions
- Only exists in memory (runtime)

JSON

```
{
  "name": "Alice",
  "age": 30,
  "active": true
}
```

- Keys MUST be in double quotes
- No functions allowed
- Used for storing & transferring data (text format)

`JSON.stringify(obj)`

JS Object → JSON string

For sending/storing

`JSON.parse(str)`

JSON string → JS Object

For using in code

Think of JSON as a 'shipping format' — you convert to JSON to send data, and convert back to use it.

BSON Data Types

MongoDB stores data as BSON (Binary JSON)

String
`"hello"` — UTF-8 text

Integer
`42` — 32-bit or 64-bit number

Double
`3.14` — Floating point number

Boolean
`true` / `false` — Boolean value

Date
`new Date()` — Date and time (stored as ms since epoch)

ObjectId
`new ObjectId()` — Unique 12-byte document identifier

Array
`[1, 2, 3]` — List of values

Object
`{ key: value }` — Embedded/nested document

Null
`null` — Null value

Binary
`BinData(0, "...")` — Binary data (e.g. images, files)

Every document automatically gets an `_id` field of type ObjectId if not provided.

BSON vs JSON — Why BSON Wins

BSON is MongoDB's storage format. Here's why it's better than plain JSON for a database:

More Data Types

JSON only has: string, number, boolean, array, object, null

BSON adds: Date, ObjectId, Binary, Int32, Int64, Decimal128, and more

Dates as Real Values

JSON stores dates as strings → `"2024-01-01"` (just text, no math)

BSON stores dates as actual timestamps → `new Date()` (sortable, comparable)

Efficient Binary Storage

BSON is binary (not text) → faster to read/write for a database engine

JSON is plain text → slower to parse at scale

Precise Numbers

JSON has one "number" type — no distinction between int and float

BSON has Int32, Int64, Double, Decimal128 → no precision loss

Unique IDs Built-in

BSON has `ObjectId` — a compact, auto-generated unique ID

JSON has no native ID type

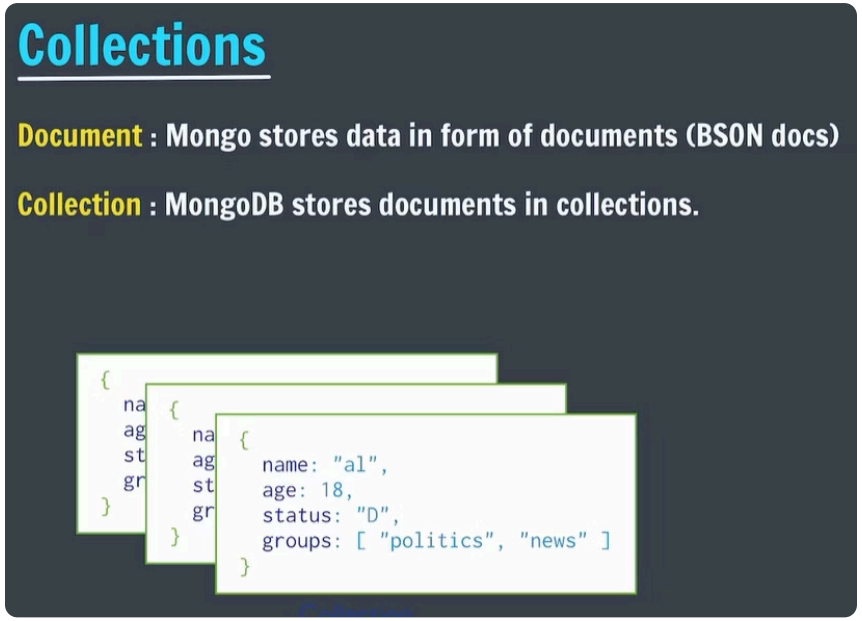
Supports Binary Data

BSON can store raw binary (images, files) directly

JSON cannot — you'd need to encode it as Base64 string first

BSON is designed for machines. JSON is designed for humans. MongoDB uses BSON internally but lets you work with JSON-like syntax.

Collections & Documents



If you know SQL, MongoDB maps directly to concepts you already know.

SQL	MongoDB	Description
Database	Database	The top-level container
Table	Collection	A group of related data
Row / Record	Document	A single entry of data
Column	Field	A single property/attribute
Primary Key	`_id`	Unique identifier per record
JOIN	Embedded Doc / `$lookup`	Linking related data

SQL Table: users

```
| id | name  | age | status |
|----|-------|-----|--------|
| 1  | Alice | 25  | active |
| 2  | Bob   | 30  | inactive |
```

MongoDB Collection: users

```
{ _id: ObjectId(), name: "Alice", age: 25, status: "active" }
{ _id: ObjectId(), name: "Bob",   age: 30, status: "inactive" }
```

Key difference: MongoDB documents in the same collection don't need to have the same fields — they're flexible, unlike SQL rows which must match the table schema.

INSERT in DB

insertOne

`db.collection.insertOne()`	Inserts a single document into a collection.

show collections

db.student.insertOne({ name: “adam”, marks: 79 })

db.student.find( )

If a collection does not exist, MongoDB creates the collection when you first store data for that collection.

```
test> use college
switched to db college
college> show collections

college> db.student.insertOne( {name: "adam", age: 19, marks: 88} )
{
  acknowledged: true,
  insertedId: ObjectId("64fec88c4d37352da5df8407")
}
college> db.student.find()
[
  {
    _id: ObjectId("64fec88c4d37352da5df8407"),
    name: 'adam',
    age: 19,
    marks: 88
  }
]
college>
```

INSERT in DB

insertMany (array of documents)

db.student.insertMany([{ name: “bob”, marks: 65 }, { name: “eve”, city: “Delhi“ }])

`db.collection.insertMany()`	Inserts multiple documents into a collection.

FIND in DB

db.collection.find( ) //returns everything

for specific queries

db.collection.find({ key: value })

db.collection.findOne({ key: value })

```
college> db.student.find( {city: "Delhi"} )
[
  {
    _id: ObjectId("64fec9714d37352da5df8408"),
    name: 'bob',
    city: 'Delhi',
    marks: 75
  },
  {
    _id: ObjectId("64fec9f54d37352da5df8409"),
    name: 'catlyn',
    marks: 64,
    city: 'Delhi'
  }
]
```

FIND in DB

Query Operators

Q. Find students where marks > 75

```
db.student.find( {marks: {$gt: 75}} )
```

Q. Find students who live in Delhi or Mumbai

```
db.student.find( {city: {$in: ["Delhi", "Mumbai"]}} )
```

Q. Find students who scored > 75 or live in Delhi

```
db.student.find( {$or: [ {marks: {$gt: 75}}, {city: "Delhi"}]} )
```

www.w3schools.com

W3Schools.com

W3Schools offers free online tutorials, references and exercises in all the major languages of the web. Covering popular subjects like HTML, CSS, JavaScript, Python, SQL, Java, and many, many more.

mongodb

Query Predicates – Database Manual – MongoDB Docs

Learn about query predicates in MongoDB. Use these expressions to filter documents in ad-hoc CRUD commands and aggregation pipelines

```
college> db.student.find({city: "Delhi", marks: 75})
[
  {
    _id: ObjectId("64fec9714d37352da5df8408"),
    name: 'bob',
    city: 'Delhi',
    marks: 75
  }
]
college> db.student.find( {marks: {$gt: 75}} )
```

Update

Updates the **first matching document** in a collection.

```
db.collection.updateOne(
  { filter },      // which document to find
  { $set: { ... } } // what to change
)
```

db.collection
The collection you want to update (e.g. `db.users`)

{ filter }
The condition to find the document (e.g. `{ name: "Alice" }`)

$set: { ... }
The fields to update (e.g. `{ age: 26 }`)

Only one document
Even if multiple match the filter

Real example

```
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26, status: "active" } }
)
```

Result

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

$set
Set/update a field value

$unset
Remove a field

$inc
Increment a number field

$push
Add item to an array

$pull
Remove item from an array

Use `updateMany()` if you want to update ALL matching documents instead of just the first one.

UPDATE in DB

Update Operators

- `$addFields`
- `$set`
- `$project`
- `$unset`
- `$replaceRoot`
- `$replaceWith`

`db.collection.updateMany()`	Update all documents that match a specified filter.
`db.collection.replaceOne()`	Replaces at most a single document that match a specified filter even though multiple documents may match the specified filter.

Nesting

```
{
  _id: ObjectId("64fdfcad780181c3d03ec623"),
  name: 'farah',
  performance: { marks: 88, grade: 'A' }
}
```

to find

```
db.student.findOne( {"performance.marks": 88} )
```

DELETE in DB

deleteOne

db.collection.deleteOne(<filter> , <options>)

deleteMany

db.collection.deleteMany(<filter> , <options>)

db.dropDatabase( )

```
college> db.student.deleteOne( {state: "Haryana"} )
{ acknowledged: true, deletedCount: 1 }
college> db.student.find()
```